

Overview

Distributed LLM training becomes mandatory when model parameters, sequence length, or throughput targets exceed single-device limits. This chapter focuses on system-level decisions: how to partition memory and compute, how to keep runs stable, and how to reason about cost-performance trade-offs under real infrastructure constraints.

For most teams, the challenge is not “which buzzword parallelism to use?” but “what is the simplest strategy that fits memory, meets throughput, and remains operable by the team?”

Parallelism Taxonomy and Formal Definitions

Data parallelism (DDP baseline)

Each worker stores a full model replica and processes distinct mini-batches. Gradients are synchronized every step.

Strength: - simple mental model, - robust ecosystem tooling.

Limitation: - full replica memory cost per worker.

Fully Sharded Data Parallel (FSDP)

FSDP shards parameters, gradients, and optimizer state across workers, reducing per-GPU memory overhead while keeping training semantics close to data parallel workflows.

Use when: - DDP hits memory limits, - model remains operator-friendly under sharding.

ZeRO (DeepSpeed)

ZeRO progressively partitions optimizer states, gradients, and parameters by stage, targeting memory efficiency at large scale.

Practical note: ZeRO-3 can unlock larger model training but increases checkpoint and runtime orchestration complexity.

Tensor and pipeline parallelism

- Tensor parallelism splits large matrix operations across devices.
- Pipeline parallelism splits model layers into stages and streams micro-batches through stages.

Both are often introduced after sharded data-parallel baselines no longer satisfy memory/throughput constraints.

Strategy Selection Framework

Stepwise decision path

1. Start with DDP baseline and profile memory/step-time.
2. Move to FSDP or ZeRO when memory is the bottleneck.
3. Add tensor parallelism if single-layer dimensions are too large.
4. Add pipeline parallelism for very deep models or multi-node balance.
5. Use hybrid patterns only when profiling proves need.

Capacity sizing heuristic

Estimate whether strategy fits by checking: - per-device memory headroom for params + grads + optimizer states, - communication bandwidth relative to expected step time, - checkpoint size and restore window constraints.

Trade-off table

Strategy	Memory relief	Comm complexity	Operational complexity	Typical stage
DDP	Low	Moderate	Low	Early scaling
FSDP	High	High	Medium	Mid-to-large
ZeRO-3	Very high	High	High	Large-scale tuning/pretraining
Tensor parallel	Medium	High	High	Very large layers
Pipeline parallel	Medium	Medium	High	Deep models / multi-node

Systems Engineering Considerations

Memory vs communication

More sharding lowers memory pressure but increases synchronization overhead. If interconnect quality is poor, theoretical gains may not materialize.

Throughput vs stability

Large effective batch sizes can improve throughput but destabilize optimization without proper learning-rate schedules, gradient clipping, and mixed-precision safeguards.

Topology-aware execution

Cluster topology (NVLink, PCIe, InfiniBand) materially changes scaling efficiency. Placement-aware scheduling and communication-aware partitioning reduce cross-node bottlenecks.

Checkpoint and recovery design

Production runs need: - resumable checkpoints, - optimizer-state continuity, - deterministic config snapshots, - conversion path to inference artifacts.

Operational Playbook

Pre-run checklist

- fixed seeds and config lock,
- explicit precision mode and overflow policy,
- memory budget test on representative sequence lengths,
- checkpoint interval + retention,
- stop criteria for divergence or exploding loss.

In-run telemetry

Track: - tokens/sec and step-time variance, - communication wait percentage, - OOM/restart incidents, - gradient norm behavior, - cost per training hour.

Post-run validation

- checkpoint integrity test and restore drill,
- evaluation gate pass/fail by critical slices,
- training report with cost-quality curves and residual risks.

Production Case Studies & Exceptions

Case 1: Enterprise domain adaptation run

Pattern: DDP baseline moved to FSDP after context-window expansion hit GPU memory limits.

Result: run fit succeeded with acceptable throughput.

Exception: weak network fabric reduced scaling efficiency, requiring worker-count retuning.

Case 2: Multi-node instruction tuning

Pattern: ZeRO-3 + gradient checkpointing for larger batch sizes.

Result: stable training throughput and higher-quality adaptation.

Exception: checkpoint management complexity increased deployment turnaround time.

Case 3: Frontier-scale training stack

Pattern: hybrid sharding + tensor parallel + pipeline staging.

Result: feasible large-model training.

Exception: debugging and fault recovery required specialized platform maturity; not suitable for small teams without platform support.

Interview Questions & Answers

1. **Q:** Why is distributed training required for LLMs?
A: Model and sequence scale exceed single-device memory/compute limits.
2. **Q:** What does FSDP shard?
A: Parameters, gradients, and optimizer states.
3. **Q:** What problem does ZeRO solve?
A: Redundant memory usage in replicated data-parallel training.
4. **Q:** Tensor vs pipeline parallelism in one line?
A: Tensor splits operations within layers; pipeline splits layers across stages.
5. **Q:** When should you start with DDP?
A: When it fits memory and provides enough throughput with lowest complexity.
6. **Q:** Key FSDP risk?
A: Communication overhead dominating training step time.
7. **Q:** Why does interconnect topology matter?
A: Distributed performance is often bounded by bandwidth/latency, not FLOPs.
8. **Q:** What are pipeline bubbles?
A: Idle stage periods caused by imperfect micro-batch scheduling.
9. **Q:** Why is checkpoint design a production concern?
A: Failed runs are expensive; recovery speed determines operational resilience.
10. **Q:** What does “hybrid parallelism” mean?
A: Combining multiple parallelism strategies in one training stack.
11. **Q:** What metric indicates scaling inefficiency?
A: Rising communication wait ratio as worker count increases.

12. **Q:** How do you improve stability at scale?
A: LR schedule tuning, clipping, mixed-precision safeguards, and gradient monitoring.
13. **Q:** Why profile before changing strategy?
A: Bottlenecks may be I/O or network, not memory.
14. **Q:** DDP vs ZeRO-3 operationally?
A: DDP is simpler; ZeRO-3 is more memory-efficient but operationally heavier.
15. **Q:** Typical anti-pattern?
A: Introducing tensor/pipeline parallelism before exhausting simpler sharding options.
16. **Q:** What metadata should every run capture?
A: Topology, sharding config, precision mode, software versions, and checkpoint lineage.
17. **Q:** How do you decide worker count?
A: Use profiling to balance throughput gains against communication overhead and cost.
18. **Q:** Why include restore drills?
A: Recovery path is part of reliability, not optional documentation.
19. **Q:** What is the cost-quality frontier in training ops?
A: The best model quality reachable within accepted infrastructure and time budget.
20. **Q:** One-line design rule?
A: Use the least complex parallel strategy that satisfies memory and throughput goals.

References

- Stanford CS336: <https://cs336.stanford.edu/>
- PyTorch FSDP docs: <https://docs.pytorch.org/docs/stable/fsdp.html>
- DeepSpeed ZeRO docs: <https://deepspeed.readthedocs.io/en/latest/zero3.html>